

Programmation Backend

Framework Laravel



Pr. ELALAOUI Hasna

h.elalaoui@uca.ac.ma

1

Introduction générale



Pattern MVC

-1

- Un des plus célèbres design patterns s'appelle MVC, qui signifie **Modèle - Vue - Contrôleur**
- Le pattern MVC permet de bien organiser son code source.
- Il va vous aider à savoir quels fichiers créer, mais surtout à définir leur rôle.
- Le but de MVC est justement de séparer la logique du code en trois parties que l'on retrouve dans des fichiers distincts comme le montre la figure suivante.



Modèle
(accès à la base de données)



Vue
(affichage de la page)



Contrôleur
(logique, calculs et décisions)



Chaque partie du modèle MVC à un rôle spécifique, ces rôles peuvent être lister comme suit :

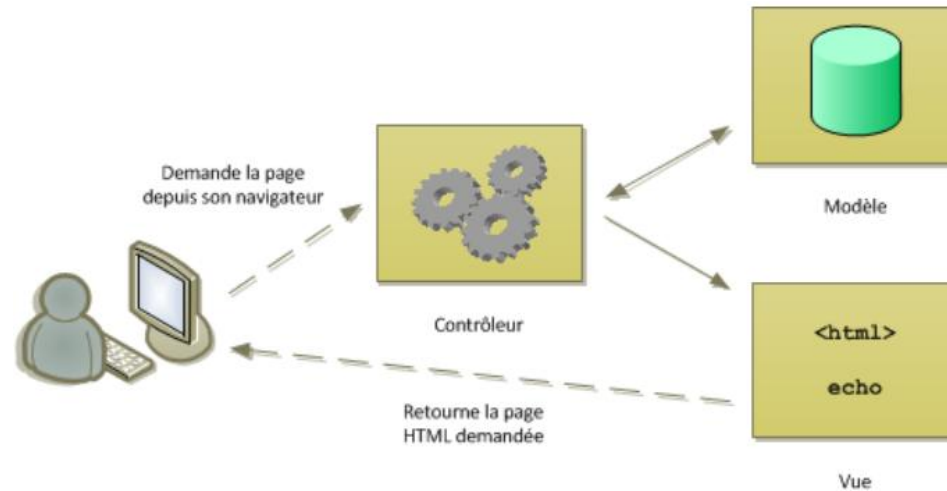
Composant	Objectif
Modèle	Son rôle est d'interagir avec la base de données par exemple : aller récupérer les informations « brutes » dans la base de données, de les organiser et de les assembler pour qu'elles puissent ensuite être traitées par le contrôleur. On y trouve donc entre autres les requêtes SQL.
Vue	cette partie se concentre sur l'affichage. Elle ne fait presque aucun calcul et se contente de récupérer des variables pour savoir ce qu'elle doit afficher. On y trouve essentiellement du code HTML mais aussi quelques boucles et conditions PHP très simples, pour afficher par exemple une liste de messages.
Contrôleur	cette partie gère la logique du code qui prend des décisions. C'est en quelque sorte l'intermédiaire entre le modèle et la vue : le contrôleur va demander au modèle les données, les analyser, prendre des décisions et renvoyer le texte à afficher à la vue. Le contrôleur contient exclusivement du PHP. C'est notamment lui qui détermine si le visiteur a le droit de voir la page ou non (gestion des droits d'accès)



Pattern MVC

-3

- En bref, la demande de l'utilisateur (exemple : requête HTTP) est reçue et interprétée par le Contrôleur.
- Celui-ci utilise les services du Modèle afin de préparer les données à afficher.
- Ensuite, le Contrôleur fournit ces données à la Vue, qui les présente à l'utilisateur (par exemple sous la forme d'une page HTML)





Un **framework** c'est quoi?

- Un **framework** désigne en programmation informatique un ensemble d'outils et de composants logiciels à la base d'un logiciel ou d'une application
- C'est le framework, qui établit les fondations d'un logiciel ou son squelette applicatif
- L'objectif du framework est de simplifier et d'uniformiser le travail des développeurs
- Il fonctionne comme un cadre ou un patron de développement

2

Framework Laravel



Le framework **Laravel**

- Laravel est un framework PHP qui a été créé par Taylor Otwell, qui initie alors une nouvelle façon de concevoir un framework en utilisant ce qui existe de mieux pour chaque fonctionnalité.
- Il a été, en ce sens, construit en se basant sur Symfony, un autre framework PHP reconnu mondialement pour sa robustesse.
- De fait, il embarque des briques logicielles testées et approuvées par une grande communauté permettant d'améliorer la rapidité des développements et de la robustesse de l'application.
- C'est un framework très complet et son utilisation est en plein essor.



Composants Laravel -1



Laravel intègre par défaut un ensemble de composants et de systèmes en cherchant à faciliter la tâche de développement aux programmeurs et ainsi réduire le temps de codage, ces modules sont principalement

Composant	Objectif
Système de routage	Laravel intègre en son sein un système de routage. Il permet de créer des chemins URLs qui sont simples et naturelles pour accéder aux différentes fonctionnalités du site en développement. Par exemple, au lieu d'avoir "index.php?article_id=57" comme URL pour accéder à la page d'un article, on pourrait mettre en place l'URL suivante : "/mon blogs/article/presentation-delaravel. Le routeur de Laravel permet de définir des URL créatives, associables à différentes zones de votre application simultanément.
Système de migration pour les bases de données	Laravel propose un outil permettant de créer et de mettre à jour un schéma de base de données. Toujours dans le même esprit qu'Eloquent et le Query Builder, il s'agit de centraliser toute la gestion de la base de données au sein de Laravel. Cet outil permet de prendre en charge tout ce qui concerne la maintenance des tables d'une base de données (création de tables, de colonnes, d'index, etc.)



Composants Laravel -2

Composant	Objectif
ORM	Un mapping objet-relationnel (en anglais object-relational mapping ou ORM) est une technique de programmation informatique qui définit des correspondances entre une base de données et les objets du langage utilisé. L'utilisation de la programmation orientée objet avec une base de données relationnelle nécessite de convertir les données relationnelles en objets et vice-versa. Ceci conduit à programmer cette conversion pour chaque objet et donc à dupliquer énormément de code similaire. C'est donc principalement à ce problème que tente de répondre un ORM. Laravel implémente son ORM avec le système Eloquent.
Moteur de template	Un moteur de template s'occupe de séparer tout le code de présentation (tout ce qui est HTML et CSS, donc la VUE du modèle MVC) et le code applicatif (le code PHP). Blade est le moteur de template proposé par Laravel. Il donne la possibilité d'utiliser des éléments communs à plusieurs pages (par exemple : header, menu, footer etc.) en agissant sur un fichier uniquement, grâce à un système d'héritage. Cela dans le but de réduire la redondance du code et d'améliorer la visibilité, la compréhension et la maintenabilité du code.



Installation et Configuration

- La dernière version stable de Laravel est Laravel 11 (sortie en 2024, toujours la version recommandée en 2026).
- Elle apporte une structure plus simple, moins de fichiers de configuration et une meilleure performance.
- Pour installer la dernière version de Laravel, on a besoin des dépendances suivantes:
 - PHP $\geq$ v8.2
 - Composer (Gestionnaire de dépendances)
 - Serveur (optionnel): XAMPP par exemple
- 2 manières d'installer:
 - Installation avec Composer (recommandée):
 - **composer create-project laravel/laravel mon_projet**
 - Installer via Laravel Installer (plus rapide):
 - **composer global require laravel/installer**
 - **laravel new mon_projet**



A Dependency Manager for PHP



Création d'un projet -1

Deux manières pour créer un projet Laravel

En utilisant composer

Composer est un outil de gestion des dépendances en PHP. Il vous permet de déclarer les bibliothèques dont votre projet dépend et il les gèrera (installation/mise à jour) pour vous.

```
composer create-project laravel/laravel  
projectName --prefer-dist
```

En utilisant l'installateur Laravel

Une autre solution pour installer Laravel consiste à utiliser l'installateur. Il faut commencer par installer globalement l'installateur avec Composer

```
Laravel new projectName
```



Création d'un projet -2

Les options à fournir:

- **Composer** : signifie qu'on utilise le Composer
- **create-project** : signifie qu'on crée un projet
- **laravel/laravel** : va chercher les fichiers à installer pour le Framework Laravel
- **projectName** : on met le nom qu'on veut donner à notre projet
- **--prefer-dist** : il y a plusieurs sous branches de Laravel, et cette commande-là va chercher la version complète de Laravel.

Une fois l'installation complétée, testez si ça a bel et bien fonctionné. Pour ce faire, ouvrez votre fenêtre de navigation, et entrez votre lien pour le site



```
localhost/projectName/public
```



Structure d'un projet -1

Une fois un projet laravel est créé, on trouve une hiérarchie de dossiers, chacun est fait pour une fonctionnalité bien déterminée. Reste à signaler que la structure d'un projet Laravel peut se différencier d'une version à l'autre.

- **Jobs:** tâches (souvent en file d'attente / queue).
- **Events:** événements nécessaires pour l'application
- **Exceptions:** ce qui concerne les exceptions
- **app/Http/:** tout ce qui est axé sur la communication, c-à-d les contrôleurs, middlewares et requêtes
- **Providers:** Configuration simplifiée et beaucoup de providers automatiquement gérés
- **Policies:** gestion des droits d'accès.
- **app/Models/User.php:** qui est un exemple de classe utilisateur pour la base de données.
- **Bootstrap:** contient les scripts d'initialisation de Laravel pour le chargement automatique des classes, les chemins pour le démarrage de l'application



Structure d'un projet -2

Depuis Laravel 11, la structure est simplifiée et certaines configurations sont centralisées.

- **Public:** C'est le point d'entrée de l'application (index.php)
- **Vendor:** qui contient tous les composants de Laravel et ses dépendances
- **Config:** qui regroupe toutes les configurations de l'application, authentification, cache, database, namespace, emails, systèmes de fichier, session...
- **Database:** pour les scripts de migration,
- **Resources:** qui contient les vues, les fichiers de langue, CSS...
- **Routes:** contenant le système de routing pour les vues, notamment le fichier web.php qui définit les routes Web (contrôleurs ou vues)
- **Storage:** qui stocke des données temporaires de l'application (vues compilées, caches, clés de session, logs...)
- **Tests:** fichiers de tests unitaires
- **Artisan:** l'outil de Laravel pour des tâches de gestion
- **composer.json:** le fichier de configuration de Composer
- **phpunit.xml:** le fichier de configuration de phpunit (pour les tests unitaires)
- **.env:** le fichier qui spécifie l'environnement d'exécution (notamment si on est en mode debug ou pas).



Création d'un projet -3

Relier la base de données:

- Maintenant, il faut relier la base de données au projet. A la racine du projet, il existe le fichier « .env ».
- En l'ouvrant, on peut voir plusieurs options de paramétrage, dont un qui est associé à la configuration de la connexion avec mysql.
- Pour '**DB_DATABASE**' => Le nom de la base de données
- Pour '**DB_USERNAME**' => Le nom d'utilisateur pour se connecter à mysql
- Pour '**DB_PASSWORD**' => le mot de passe.



Structure d'un projet

Récapitulatif

```
▼ MON_PROJET
  > app
  > bootstrap
  > config
  > database
  > public
  ▼ resources
    > css
    > js
    > views
  > routes
  > storage
  > tests
  > vendor
  ⚙ .editorconfig
  ⚙ .env
  ⚙ .env.example
  💎 .gitattributes
  💎 .gitignore
  ≡ artisan
  {} composer.json
  {} composer.lock
  {} package.json
  📄 phpunit.xml
  ⓘ README.md
  ⚡ vite.config.js
```

- `app` → logique
- `routes` → navigation
- `resources/views` → affichage
- `public` → point d'entrée
- `config` → paramètres et configuration
- `vendor` → bibliothèques

3

Modèle et Migration



Migration -1

Présentation

- Les migrations sont comme un contrôle de version pour votre base de données, permettant à votre équipe de modifier et de partager le schéma de la base de données de l'application.
- Les migrations sont généralement couplées avec le constructeur de schémas de Laravel pour construire le schéma de la base de données de votre application.
- Si vous avez déjà dû dire à un coéquipier d'ajouter manuellement une colonne à son schéma de base de données local, vous avez été confronté au problème que les migrations de base de données résolvent.

Création

- Pour créer une migration, il faut utiliser la commande `make:migration` du gestionnaire Artisan

```
php artisan make:migration create_nomTables_table
```

- La nouvelle migration sera placée dans le répertoire `database/migrations`. Chaque nom de fichier de migration contient un horodatage, qui permet à Laravel de déterminer l'ordre des migrations (historique de modification de la BD).



Migration -2

Structure

- Une classe de migration contient deux méthodes : la méthode « UP » et la méthode « DOWN ».
- La méthode « **up** » est utilisée pour ajouter de nouvelles tables, colonnes ou index à votre base de données, tandis que la méthode « **down** » fait l'inverse des opérations effectuées par la méthode « up ».

Exécution

- Pour exécuter toutes les migrations en cours, il faut exécuter la commande migrate d'Artisan.
- Dans ce cas la méthode « up » s'exécute

```
php artisan migrate
```

Annulation

- Pour annuler la dernière opération de migration, vous pouvez utiliser la commande **rollback**
- Dans ce cas la méthode « down » s'exécute

```
php artisan migrate:rollback
```



Migration -3

Créer une table



Pour créer une nouvelle table de base de données, utilisez la méthode « **create** » de la façade du schéma. La méthode de création accepte deux arguments : le premier est le nom de la table, tandis que le second est un **Closure** qui reçoit un objet **Blueprint** qui peut être utilisé pour définir la nouvelle table :

```
public function up()
{
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->float('price');
    });
}

public function down()
{
    Schema::dropIfExists('products');
}
```



Migration -4

Créer des colonnes de table

- La méthode « **table** » de la façade du schéma peut être utilisée pour mettre à jour les tables existantes.
- Tout comme la méthode de création, la méthode de table accepte deux arguments : le nom de la table et une fermeture qui reçoit une instance de Schéma que vous pouvez utiliser pour ajouter des colonnes à la table :

```
Schema::table('nomDeTable', function (Blueprint $table) {  
    $table->typeDeChamps('nomDuChamps');  
});
```

- Une liste exhaustive des types supportés par la dernière version de Laravel se trouve sur le lien : <https://laravel.com/docs/13.x/migrations>



Migration -5

Créer des indexes

- Le constructeur de schémas Laravel prend en charge plusieurs types d'index. L'exemple suivant crée une nouvelle colonne de courrier électronique et précise que ses valeurs doivent être uniques.
- Pour créer l'index, nous pouvons enchaîner la méthode « unique » sur la définition de la colonne :

```
$table->string('email')->unique();
```

- Ou bien après la création de la colonne « email »

```
$table->unique('email');
```



Modèle

Présentation générale

- L'ORM Eloquent inclut avec Laravel offre une implémentation simple pour exploiter une base de données.
- Chaque table de base de données a un "**modèle**" correspondant qui est utilisé pour interagir avec elle.
- Les modèles permettent de rechercher des données dans vos tables, ainsi que d'insérer de nouveaux enregistrements dans la table.

Création de modèles

- La façon la plus simple de créer une instance de modèle est d'utiliser la commande `make:model` d'Artisan

```
php artisan make:model nomDuModele
```

- Si vous souhaitez générer une migration de base de données lorsque vous générez le modèle, vous pouvez utiliser les boutons `--migration` ou `-m`

```
php artisan make:model nomDuModele -m
```




Conventions

Noms des tables

- Notez que nous n'avons pas dit à Eloquent quelle table utiliser pour notre modèle.
- Par convention, le nom pluriel de la classe, en "**snake case**", sera utilisé comme nom de table à moins qu'un autre nom ne soit explicitement spécifié.
- Ainsi, dans ce cas, Eloquent supposera que le modèle stocke les enregistrements dans la table concernée.

Clés primaires

- Eloquent supposera également que chaque tableau possède une colonne de clé primaire nommée **id**. Vous pouvez définir une propriété **\$primaryKey** protégée pour passer outre à cette convention
- En outre, Eloquent suppose que la clé primaire est une valeur entière incrémentielle, ce qui signifie que par défaut la clé primaire sera automatiquement coulée à un int.
- Si vous souhaitez utiliser une clé primaire non incrémentielle ou non numérique, vous devez définir la propriété publique **\$incrementing** de votre modèle à false



Récupération des données -1

- Une fois que vous avez créé un modèle et sa table dans la base de données associée, vous êtes prêts à commencer à récupérer des données dans votre base de données.
- Considérez chaque modèle Eloquent comme un puissant constructeur de requêtes vous permettant d'interroger couramment la table de base de données associée au modèle.

Exemple :

```
$cars = App\Car::all();  
foreach ($cars as $car) {  
    echo $car->marque;  
}
```

- La méthode « all » d'Eloquent renvoie tous les résultats dans le tableau du modèle



Récupération des données -2



Comme chaque modèle Eloquent sert comme constructeur de requêtes, vous pouvez également ajouter des contraintes aux requêtes, puis utiliser la méthode "**get**" pour récupérer les résultats par exemple :

Exemple

```
$cars = App\Car::where('marque', 'BMW')  
->orderBy('name', 'desc')  
->take(10)  
->get();
```



Récupération des données -3

- En plus de récupérer tous les enregistrements d'une table donnée, vous pouvez également récupérer des enregistrements individuels en utilisant les fonctions `find`, `first`, ou `firstWhere`.
- Au lieu de renvoyer une collection de modèles, ces méthodes renvoient une seule instance de modèle

Exemple

```
// Récupération par clé primaire...  
$car = App\Car::find(1);  
  
// Récupération par regroupement de critères...  
$car = App\Car::where('active', 1)->first();  
  
// Retourne le premier enregistrement répondant au critère...  
$car = App\Car::firstWhere('active', 1);
```



Insertion des données



Pour créer un nouvel enregistrement dans la base de données, créez une nouvelle instance de modèle, définissez des attributs sur le modèle, puis appelez la méthode de sauvegarde **save()** :

```
$car = new Car;  
$car->marque = $request->marque;  
$car->save();
```



Dans cet exemple, nous attribuons le paramètre marque de la requête HTTP entrante à l'attribut marque de l'instance du modèle App\Car.



Lorsque nous appelons la méthode **save()**, un enregistrement sera inséré dans la base de données.



Les timestamps **created_at** et **updated_at** seront automatiquement définis lors de l'appel de la méthode de sauvegarde, il n'est donc pas nécessaire de les définir manuellement.



Modification des données

- La méthode **save()** peut également être utilisée pour mettre à jour les modèles qui existent déjà dans la base de données.
- Pour mettre à jour un modèle, vous devez le récupérer, définir les attributs que vous souhaitez mettre à jour, puis appeler la méthode de sauvegarde.
- Là encore, l'horodatage `updated_at` sera automatiquement mis à jour, il n'est donc pas nécessaire de définir manuellement sa valeur

```
$car = App\Car::find(1);  
$car->marque = 'Audi';  
$car->save();
```



Suppression des données



Pour supprimer un modèle, appelez la méthode de suppression **delete()** sur une instance de modèle

```
$car = App\Car::find(1);  
$car->delete();
```



Dans l'exemple ci-dessus, nous récupérons le modèle dans la base de données avant d'appeler la méthode de suppression.



Cependant, si vous connaissez la clé primaire du modèle, vous pouvez supprimer le modèle sans le récupérer en appelant la méthode de destruction **destroy()**.



En plus d'une seule clé primaire comme argument, la méthode de destruction acceptera plusieurs clés primaires, un tableau de clés primaires ou une collection de clés primaires

```
App\Car::destroy(1);  
App\Car::destroy(1,2,3);  
App\Car::destroy([1,3,5]);
```

4

Système de routage



Présentation générale

- Tous les itinéraires Laravel sont définis dans vos fichiers d'itinéraires, qui se trouvent dans le répertoire des itinéraires.
- Ces fichiers sont automatiquement chargés par le framework. Le fichier routes/web.php définit les routes qui sont pour votre interface web.
- Ces routes sont affectées au groupe de middleware web, qui fournit des fonctionnalités comme l'état de session et la protection CSRF.
- Pour la plupart des applications, vous commencerez par définir des itinéraires dans votre fichier routes/web.php. Les routes définies dans routes/web.php sont accessibles en entrant l'URL de la route définie dans votre navigateur.

```
Route::méthode_d'envoi ('requête', 'méthode_de_réponse') ;
```



Exemple routage

- Par exemple, vous pouvez accéder à l'itinéraire suivant en naviguant sur <http://your-app.test/user> dans votre navigateur.
- Dans cet exemple nous lions une requête à une méthode nommée « index » du contrôleur « UserController »

```
Route::get('/user', [UserController::class, 'index']);
```

- get : désigne la méthode utilisée pour envoyer les données
- '/user', la valeur de la requête
- UserController : le contrôleur censé répondre à la requête
- index : le nom de la méthode qui sera déclenchée



Protection CSRF

- Tout formulaire HTML pointant sur des itinéraires POST, PUT, PATCH, ou DELETE qui sont définis dans le fichier d'itinéraires Web doit inclure un champ de jeton CSRF.
- Dans le cas contraire, la demande sera rejetée

```
<form method="POST" action="/profile">  
    @csrf  
    ...  
</form>
```



Retour de vue

- Si votre itinéraire ne nécessite qu'un retour de vue, vous pouvez utiliser la méthode **Route::view**.
- Cette méthode fournit un simple raccourci de sorte que vous n'avez pas à définir un itinéraire complet ou un contrôleur.
- La méthode de vue accepte un URI comme premier argument et un nom de vue comme deuxième argument.
- En outre, vous pouvez fournir un tableau de données à transmettre à la vue comme troisième argument facultatif :

```
Route::view( 'URI', 'nom_de_la_vue' );
```



Route avec paramètres -1



Parfois, vous devrez capturer des segments de l'URI dans votre itinéraire. Par exemple, vous pouvez avoir besoin de capturer l'ID d'un utilisateur à partir de l'URL. Vous pouvez le faire en définissant les paramètres de l'itinéraire.

Exemple

```
Route::get('user/{id}', function ($id) {  
    return 'User '.$id;  
});
```



Vous pouvez définir autant de paramètres d'itinéraire que l'exige votre itinéraire.

Exemple

```
Route::get('posts/{post}/comments/{comment}', function ($postId, $commentId) {  
    // traitement  
});
```



Route avec paramètres -2

- Les paramètres d'itinéraire sont toujours encadrés par des accolades {} et doivent être constitués de caractères alphabétiques, et ne peuvent pas contenir de caractère -.
- Au lieu d'utiliser le caractère -, utilisez un trait de soulignement (_).
- Les paramètres de route sont injectés dans les callbacks / contrôleurs de route en fonction de leur ordre - les noms des arguments des callbacks / contrôleurs n'ont pas d'importance.



Les controllers

- Au lieu de définir toute votre logique de traitement des demandes dans les fichiers de routage, vous pouvez organiser ce comportement en utilisant des classes de contrôleur.
- Les contrôleurs peuvent regrouper les logiques de traitement des demandes connexes dans une seule classe.
- Les contrôleurs sont stockés dans le répertoire **app/Http/Controllers**.
- Afin de créer un contrôleur, la syntaxe est la suivante

```
php artisan make:controller nom_du_controleur
```

- Par convention les noms des classes contrôleurs sont suffixés par « **Controller** »
- Pour créer un contrôleur dit Ressource, il faut intégrer l'option `--resource`

```
php artisan make:controller nom_du_controleur --resource
```

5

Moteur de template



Moteur Blade

- Blade est le moteur de vue utilisé par Laravel. Il offre une syntaxe simplifiée pour exécuter de petits bouts de code dans une vue, par exemple pour vérifier une condition avant d'afficher une information ou encore pour boucler dans une collection.
- Par exemple, il serait possible de boucler dans une collection à l'aide de la syntaxe PHP :

Sans l'utilisation de Blade

```
<?php foreach($produits as $produit) : ?>
    <li><?= $produit->description;?></li>
<?php endforeach; ?>
```

Avec l'utilisation de Blade

```
@foreach($produits as $produit)
    <li>{{ $produit->description }}</li>
@endforeach
```

- Il suffit d'ajouter « blade » avant l'extension « php » : `pageName.blade.php`



Les commentaires

- À la base, tout ce que vous écrivez dans une vue sera transmis tel quel par Blade au navigateur. Vous pouvez donc entrer sans plus de blabla les balises HTML et le texte statique.
- Il existe cependant des exceptions afin de permettre d'entrer des commentaires, d'afficher des variables, de tester des conditions et de faire des boucles.

Commentaires

- Lorsque vous ajoutez un commentaire avec Blade, vous êtes assurés qu'il ne fera pas partie du code HTML généré

```
{{-- Commentaire --}}
```

Fonctions PHP

- Il est possible d'exécuter des fonctions PHP en les plaçant entre les symboles {{ et }}.

```
{{ fonctionName(parameters) }}
```

Variables

- Pour afficher le contenu d'une variable (que le contrôleur a transmis à la vue), simplement placer son nom entre {{ et }}.

```
{{ $variableName }}
```



```
<?php echo $variableName; ?>
```



Les structures de **contrôle**

Tester une condition

```
@if (...)  
    ...  
@elseif (...)  
    ...  
@else  
    ...  
@endif
```

Boucler les éléments d'une collection

```
@foreach($tableau as $variable)  
    ...  
@endforeach
```



Les directives PHP

- Dans la majorité des cas, la logique de l'application sera faite dans le contrôleur ou dans un autre fichier spécifique. La vue ne fera que de l'affichage avec possiblement quelques structures de contrôle.
- Il existe pourtant des cas précis où le code sera plus clair si on ajoute un peu de PHP, par exemple l'initialisation d'une variable qui clarifiera le code qui suit.
- Pour cela, il faut utiliser la directive `@php`.

```
@php
```

```
    $uneVariable = ...;
```

```
@endphp
```



Les assets

- Pour charger les scripts et les feuilles de styles, on utilise l'helper « **asset** ».
- Celui-ci pointe directement vers le dossier **public**. Il faut donc l'utiliser au maximum afin d'éviter les éventuelles erreurs dues au changement d'URL.
- Exemple, nous voulons appeler le script jquery.js se trouvant dans **public/js** :

```
<script src="{{asset('js/jquery.js')}}"> </script>
```



Les routes

- Laravel propose une multitude d'helpers.
- Mis à part le « asset », il y a un autre à connaître, à savoir « route ». Grâce à lui, on peut générer les URLs des liens de manière dynamique.
- Si un jour une de vos URL change, les liens fonctionneront toujours si on veut utiliser cet helper.

```
<a href="{ {route('nomDeLaRoute')} } ">
```



Héritage de modèles -1

- Deux principaux avantages de l'utilisation de Blade sont l'**héritage des modèles** et les **sections**.
- Pour commencer, prenons un exemple simple. Tout d'abord, nous allons examiner une mise en page "maître". Comme la plupart des applications web conservent la même disposition générale sur plusieurs pages, il est pratique de définir cette disposition comme une vue unique de Blade

```
<!-- Stored in resources/views/layouts/app.blade.php -->
<html>
<head>
<title>App Name - @yield('title')</title>
</head>
<body>
    @section('sidebar')
        This is the master sidebar.
    @show
    <div class="container"> @yield('content')
</div>
</body>
</html>
```



Héritage de modèles -2

- Comme vous pouvez le voir, ce fichier contient un balisage HTML typique. Cependant, prenez note des directives `@section` et `@yield`. La directive `@section`, comme son nom l'indique, définit une section de contenu, tandis que la directive `@yield` est utilisée pour afficher le contenu d'une section donnée.
- Maintenant que nous avons défini une mise en page pour notre application, définissons une page enfant qui hérite de la mise en page.
- Lorsque vous définissez une vue fille, utilisez la directive Blade `@extends` pour spécifier la disposition dont la vue enfant doit "hériter".
- Les vues qui étendent une disposition de Blade peuvent injecter du contenu dans les sections de la disposition en utilisant les directives `@section`. Rappelez-vous, comme dans l'exemple traité, le contenu de ces sections sera affiché dans le layout en utilisant `@yield`



Héritage de modèles -2

- Dans cet exemple, la section de la barre latérale utilise la directive **@parent** pour ajouter (plutôt que d'écraser) le contenu de la barre latérale de la mise en page. La directive **@parent** sera remplacée par le contenu de la mise en page lorsque la vue sera rendue
- Contrairement à l'exemple précédent, cette section de la barre latérale se termine par **@endsection** au lieu de **@show**.
- La directive **@endsection** ne définira qu'une section tandis que **@show** définira et produira immédiatement la section.

```
<!-- Stored in resources/views/child.blade.php -->
@extends('layouts.app')
@section('title', 'Page Title')
@section('sidebar')
    @parent
    <p>This is appended to the master sidebar.</p>
@endsection
@section('content')
    <p>This is my body content.</p>
@endsection
```